SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – APPLICATION BASED QUESTIONS

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO2	**Assessment:**	Application Based Questions, In-class
Weightage:	40% of CIA	**Duration / Marks:**	60 mins
CO2 Statement:	Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)		
Student Name:	Purushothaman.V	**Reg. No.:**	192511064
Section / Batch:	CSE	**Date:**	21/10/2026

Code of Conduct

I Purushothaman.V / 192511064 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Purushothaman.V

Instructions

- Read each question carefully before answering (in-Class Assessment)
- Each question is a real-world application scenario. Identify the AI technique that fits, apply it step by step, and justify every decision.
- Questions are drawn from Informed Search (Greedy, A*, heuristics), Local Search and Online Search, Constraint Satisfaction Problems, and Adversarial Search (Minimax, Alpha-Beta, evaluation functions).
- Show intermediate tables (open lists, domains, α/β values) and state the final answer clearly.
- Marks will be awarded using the Application Based Questions rubric: understanding of problem context, application of concepts, problem-solving approach, accuracy of solution and clarity.
- Upload your work in GCR and Github.

Section / Topic	Focus	Question Nos.
1. Informed Search Strategies	Greedy best-first search, A* search, heuristic functions (admissibility, consistency)	**Q1**
2. Local Search, Online Search & Constraint Satisfaction	Local search and optimization problems, online search agent, CSP (MRV, degree heuristic, forward checking)	**Q2, Q3**
3. Game Playing: Adversarial Search	Games, minimax algorithm, alpha-beta pruning, evaluation functions	**Q4**

Rubrics for Application Based Questions

Criteria	Weightage	Level 4 – Exemplary (100% of marks)	Level 3 – Proficient (75%)	Level 2 – Developing (50%)	Level 1 – Beginning (25%)
Understanding of Problem Context	**20%**	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	**30%**	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	**20%**	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	**20%**	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	**10%**	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context (2)	Application of Concepts (3)	Problem-Solving Approach (2)	Accuracy of Solution (2)	Clarity (1)	Sub. Total (10)	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1: Campus Delivery Robot – Greedy Best-First vs A*

Topic: Informed search strategies, heuristic functions | 15 minutes

A campus delivery robot must carry a parcel from the Store (S) to the Hostel (G). The one-way road network and the robot's heuristic estimate h(n) of the remaining travel cost to G (in minutes) are given below.

From	To	Cost
S	A	1
S	B	4
A	C	2
A	D	5
B	G	5
C	D	1
D	G	2

Node	S	A	B	C	D	G
h(n)	6	5	3	3	2	0

Assume ties are broken alphabetically and that a node is re-opened if a cheaper path to it is found.

(a) Apply Greedy Best-First Search from S to G. Write the order in which nodes are expanded, the path returned and its total cost.

(b) Apply A* search. Show the open list with g(n), h(n) and f(n) after each expansion, the order of expansion, and the final path and cost.

(c) Check whether h(n) is admissible and consistent by verifying the required condition for every node and every road.

(d) Compare the outputs of (a) and (b). Explain why Greedy Best-First Search returned a different result and which algorithm you would deploy for the robot, with justification.

Question 1: Answer

(a) Greedy Best-First Search

Formula: Select the node with the lowest heuristic value, h(n).

Steps

1. Start at S. OPEN = {A(5), B(3)}.
2. Select B because h(B) = 3 is less than h(A) = 5.
3. Expand B and generate G with h(G) = 0.
4. Select G and terminate.

Expansion table

Step	Expanded node	OPEN list after expansion
1	S	B(3), A(5)
2	B	G(0), A(5)
3	G	Goal reached

Expansion order: S → B → G

Final path: S → B → G

Total cost:

$$4 + 5 = \boxed{9}$$

(b) A* Search

Formula:

$$\boxed{f(n) = g(n) + h(n)}$$

Where:

- $g(n)$= cost from the start node.
- $h(n)$= estimated cost to the goal.
- $f(n)$= estimated total path cost.

Step 1: Expand S

For A:

$$g(A) = 1,\ h(A) = 5$$
$$f(A) = 1 + 5 = 6$$

For B:

$$g(B) = 4,\ h(B) = 3$$

$$f(B) = 4 + 3 = 7$$

OPEN list:

Node	g(n)	h(n)	f(n)
A	1	5	6
B	4	3	7

Select A.

Step 2: Expand A

Generate C and D.

For C:

$$g(C) = 1 + 2 = 3$$
$$f(C) = 3 + 3 = 6$$

For D:

$$g(D) = 1 + 5 = 6$$
$$f(D) = 6 + 2 = 8$$

OPEN list:

Node	g(n)	h(n)	f(n)
C	3	3	6
B	4	3	7
D	6	2	8

Select C.

Step 3: Expand C

C → D has a cost of 1.

The new cost to D is:

$$g(D) = 3 + 1 = 4$$
$$f(D) = 4 + 2 = 6$$

This is cheaper than the previous cost of 6, so update D.

OPEN list:

Node	g(n)	h(n)	f(n)
D	4	2	6
B	4	3	7

Select D.

Step 4: Expand D

D → G has a cost of 2.

$$g(G) = 4 + 2 = 6$$
$$f(G) = 6 + 0 = 6$$

OPEN list:

Node	g(n)	h(n)	f(n)
G	6	0	6
B	4	3	7

Select G and terminate.

Final answer

Expansion order: S → A → C → D → G

Final path: S → A → C → D → G

Total cost:

$$1 + 2 + 1 + 2 = \boxed{6}$$

(c) Check admissibility and consistency

Admissibility

A heuristic is admissible if:

$$h(n) \leq h^*(n)$$

Here, $h^*(n)$is the actual minimum cost from node n to G.

Calculate the actual minimum costs:

Node	Actual minimum cost $h^*(n)$	Given h(n)	Admissible?
S	6	6	Yes
A	5	5	Yes
B	5	3	Yes

Node	Actual minimum cost $h^*(n)$	Given h(n)	Admissible?
C	3	3	Yes
D	2	2	Yes
G	0	0	Yes

Therefore, the heuristic is **admissible**.

Consistency

A heuristic is consistent if:

$$\boxed{h(n) \leq c(n, n') + h(n')}$$

Check every edge:

Edge	Check	Result
S → A	6 ≤ 1 + 5 = 6	Consistent
S → B	6 ≤ 4 + 3 = 7	Consistent
A → C	5 ≤ 2 + 3 = 5	Consistent
A → D	5 ≤ 5 + 2 = 7	Consistent
B → G	3 ≤ 5 + 0 = 5	Consistent
C → D	3 ≤ 1 + 2 = 3	Consistent
D → G	2 ≤ 2 + 0 = 2	Consistent

Therefore, the heuristic is **consistent**.

(d) Comparison and justification

Greedy Best-First Search	A* Search
Uses h(n).	Uses g(n) + h(n).
Considers estimated distance to the goal.	Considers the cost already travelled and the estimated remaining cost.
Returns path S → B → G.	Returns path S → A → C → D → G.
Total cost = 9.	Total cost = 6.
Does not guarantee an optimal path in general.	Finds an optimal path with an admissible heuristic under the standard graph-search conditions.

Conclusion: I would deploy A* because it considers both the actual travel cost and the estimated remaining cost. It finds the shortest path with a total cost of 6 minutes, whereas Greedy Best-First Search returns a path costing 9 minutes.

Question 2: Wi-Fi Access Point Placement – Local Search and Online Search

Topic: Local search algorithms and optimization problems, online search agent | 15 minutes

A technician wants to place a Wi-Fi access point at the best position along a corridor with 10 candidate positions x = 0 to 9. The measured signal quality f(x) (higher is better) is:

x	0	1	2	3	4	5	6	7	8	9
f(x)	2	5	9	6	4	7	12	10	8	3

A move shifts the access point to an adjacent position ($x - 1$ or $x + 1$). Steepest-ascent hill climbing moves to the better neighbour and stops when no neighbour is strictly better.

(a) Run steepest-ascent hill climbing from the start positions x = 0, x = 4 and x = 9. Show each step and the final position and signal quality.

(b) Identify all local maxima and the global maximum. If the start position is chosen uniformly at random from the 10 positions, find the probability that one run reaches the global maximum, the probability that at least one of 3 random-restart runs does, and the expected number of runs needed.

(c) A simulated annealing search is at x = 2 and proposes the move to x = 3. Compute the acceptance probability $e^{(\Delta/T)}$ at T = 2 and at T = 1, and explain what this does to the chance of escaping the local maximum as T falls.

(d) The same search is later done by a robot that cannot see the whole corridor, can only measure f(x) where it stands, and cannot undo exploration for free. Explain why this is an online search problem, and state what the agent must remember to avoid cycling.

Question 2: Answer

(a) Steepest-ascent hill climbing

Hill climbing moves to a better adjacent position and stops when neither neighbour has a strictly higher value.

Starting position x = 0

Current position	Signal quality	Neighbour comparison	Action
0	2	f(1) = 5 > 2	Move to 1
1	5	f(2) = 9 > 5	Move to 2
2	9	f(3) = 6 < 9	Stop

Final position: x = 2

Final signal quality: 9

Starting position x = 4

Current position	Signal quality	Neighbour comparison	Action
4	4	f(3) = 6, f(5) = 7	Move to 5
5	7	f(6) = 12 > 7	Move to 6
6	12	f(5) = 7, f(7) = 10	Stop

Final position: x = 6

Final signal quality: 12

Starting position x = 9

Current position	Signal quality	Neighbour comparison	Action
9	3	f(8) = 8 > 3	Move to 8
8	8	f(7) = 10 > 8	Move to 7
7	10	f(6) = 12 > 10	Move to 6
6	12	f(5) = 7, f(7) = 10	Stop

Final position: x = 6

Final signal quality: 12

(b) Local maxima, global maximum and probabilities

Local maxima

A local maximum has a higher value than its adjacent positions.

At x = 2:

$$f(2) = 9 > f(1) = 5,\ f(2) = 9 > f(3) = 6$$

Therefore, x = 2 is a local maximum.

At x = 6:

$$f(6) = 12 > f(5) = 7,\ f(6) = 12 > f(7) = 10$$

Therefore, x = 6 is also a local maximum.

All local maxima: x = 2 and x = 6.

Global maximum: x = 6, with signal quality 12.

Probability that one run reaches the global maximum

The starting positions that reach x = 6 are:

$$\{4, 5, 6, 7, 8, 9\}$$

There are 6 successful starting positions out of 10.

$$P(\text{success}) = \frac{6}{10}$$

$$\boxed{P(\text{success}) = 0.6 = 60\%}$$

Probability that at least one of 3 random-restart runs succeeds

The probability that one run fails is:

$$P(\text{failure}) = 1 - 0.6 = 0.4$$

The probability that all three runs fail is:

$$P(\text{all fail}) = (0.4)^3 = 0.064$$

Therefore:

$$P(\text{at least one success}) = 1 - 0.064$$

$$\boxed{0.936 = 93.6\%}$$

Expected number of runs needed

For independent runs with success probability $p = 0.6$, the expected number of runs until the first success follows the geometric distribution.

$$E(N) = \frac{1}{p}$$

$$E(N) = \frac{1}{0.6}$$

$$\boxed{E(N) = 1.67 \text{ runs approximately}}$$

(c) Simulated annealing

The search is at x = 2 and proposes moving to x = 3.

Given:

$$f(2) = 9,\ f(3) = 6$$

The change in signal quality is:

$$\Delta = f(3) - f(2) = 6 - 9 = -3$$

Acceptance probability:

$$\boxed{P = e^{\Delta/T}}$$

At T = 2

$$P = e^{-3/2}$$

$$\boxed{P \approx 0.2231 = 22.31\%}$$

At T = 1

$$P = e^{-3/1}$$

$$\boxed{P \approx 0.0498 = 4.98\%}$$

Explanation

As temperature decreases, the probability of accepting a worse move decreases.

At T = 2, the worse move has an acceptance probability of approximately 22.31%.

At T = 1, the probability decreases to approximately 4.98%.

Therefore, higher temperatures allow more exploration, while lower temperatures encourage the algorithm to settle into a good solution.

(d) Online search problem

This is an online search problem because the robot cannot observe the entire corridor in advance. It can measure the signal quality only at its current position.

The robot must:

1. Observe the signal quality at its current position.
2. Identify and evaluate the available neighbouring positions.
3. Move to a selected position.
4. Update its knowledge using the new observation.

To avoid cycling, the robot should remember:

- Previously visited positions.
- The signal quality measured at each visited position.
- The actions taken and the positions already explored.

This information helps the robot avoid repeatedly visiting the same positions and supports more efficient exploration.

Question 3: Exam Timetable Scheduling – Constraint Satisfaction Problem

Topic: Constraint satisfaction problem (MRV, degree heuristic, forward checking) | 15 minutes

Five exams A, B, C, D and E must each be given one slot from {1, 2, 3}. Two exams that share students must not be in the same slot. The pairs sharing students are: A–B, A–C, A–D, B–C, B–D, C–E, D–E. Because of invigilator availability, exam A can only be in slots 1 or 2, and exam E only in slots 1 or 2.

Exam	Initial domain
A	{1, 2}
B	{1, 2, 3}
C	{1, 2, 3}
D	{1, 2, 3}
E	{1, 2}

Rules for the search: choose the variable using Minimum Remaining Values (MRV), break ties with the degree heuristic, then alphabetically; try values in ascending order; use forward checking after every assignment.

(a) Draw the constraint graph (list its edges) and compute the degree of every exam.

(b) Select the first exam to assign using MRV and the degree heuristic, justify the choice, assign its first value, and show the domains of all exams after forward checking.

(c) Continue the search with the same rules, showing the chosen exam, its value and the updated domains at each step, until a complete assignment is found. Verify it against every constraint.

(d) Interpret: explain how MRV, the degree heuristic and forward checking reduced the search effort here, and what forward checking would signal if a domain became empty.

Question 3: Answer

(a) Constraint graph and degrees

The constraint graph contains five vertices: A, B, C, D and E.

Its edges are:

$$\boxed{\{A, B, A, C, A, D, BCBDCEDE\}}$$

The degree of a variable is the number of variables connected to it.

Exam	Connected exams	Degree
A	B, C, D	3
B	A, C, D	3
C	A, B, E	3
D	A, B, E	3
E	C, D	2

Therefore:

$$\boxed{\deg(A) = 3, \deg(B) = 3, \deg(C) = 3, \deg(D) = 3, \deg(E) = 2}$$

(b) Select the first exam using MRV and degree heuristic

Minimum Remaining Values (MRV): Select the exam with the smallest domain.

Initially:

- A has 2 values.
- B has 3 values.
- C has 3 values.
- D has 3 values.
- E has 2 values.

MRV selects either A or E.

Their degrees are:

$$\deg(A) = 3,\ \deg(E) = 2$$

The degree heuristic selects A because it has more neighbouring exams.

Therefore, the first variable is:

$$\boxed{A}$$

Try values in ascending order. Assign:

$$\boxed{A = 1}$$

Forward checking

Since A shares students with B, C and D, none of those exams can use slot 1.

Updated domains:

Exam	Domain before assignment	Domain after forward checking
A	{1, 2}	{1}
B	{1, 2, 3}	{2, 3}
C	{1, 2, 3}	{2, 3}
D	{1, 2, 3}	{2, 3}
E	{1, 2}	{1, 2}

(c) Continue the search

Step 1: Assign A = 1

Current domains:

- B = {2, 3}
- C = {2, 3}
- D = {2, 3}
- E = {1, 2}

Step 2: Select the next exam

B, C and D each have two values remaining. E also has two values.

Apply the degree heuristic:

- B has degree 3.
- C has degree 3.
- D has degree 3.
- E has degree 2.

Among B, C and D, select alphabetically:

$$\boxed{B}$$

Try the lowest value:

$$\boxed{B = 2}$$

Forward checking removes slot 2 from C and D because B is connected to both.

Updated domains:

Exam	Domain
A	{1}
B	{2}
C	{3}
D	{3}
E	{1, 2}

C and D become singletons because their domains were {2, 3} before assigning B.

Step 3: Assign C = 3

C has one remaining value:

$$\boxed{C = 3}$$

C and E share students, so E cannot use slot 3. However, E's domain is already {1, 2}, so no further change occurs.

Updated domains:

Exam	Domain
A	{1}
B	{2}
C	{3}
D	{3}
E	{1, 2}

Step 4: Assign D = 3

D has one remaining value:

$$\boxed{D = 3}$$

D and E share students, so E cannot use slot 3. Its domain remains {1, 2}.

Step 5: Assign E = 1

E has two values remaining, {1, 2}. Assign the lowest value:

$$\boxed{E = 1}$$

All exams now have assigned slots.

Final timetable

Exam	Assigned slot
A	1
B	2
C	3
D	3
E	1

Final assignment:

$$\boxed{A = 1, B = 2, C = 3, D = 3, E = 1}$$

Verify all constraints

Constraint	Assigned slots	Valid?
A–B	1 and 2	Yes
A–C	1 and 3	Yes
A–D	1 and 3	Yes
B–C	2 and 3	Yes
B–D	2 and 3	Yes
C–E	3 and 1	Yes
D–E	3 and 1	Yes

All constraints are satisfied, and A and E use only their permitted slots.

(d) Interpretation

MRV: MRV selects the variable with the fewest remaining values. This helps identify the most constrained exam first and reduces unnecessary search.

Degree heuristic: When MRV produces a tie, the degree heuristic selects the exam connected to the greatest number of unassigned variables. This helps resolve the most restrictive variables earlier.

Forward checking: Forward checking removes values that conflict with each assignment from the domains of neighbouring variables. This detects possible conflicts early and reduces the search space.

If a domain becomes empty: The current assignment cannot lead to a valid solution. The algorithm must backtrack and try another value for a previously assigned variable.

Question 4: Board-Game AI – Minimax, Alpha-Beta Pruning and Evaluation Function

Topic: Game playing, adversarial search, evaluation functions | 15 minutes

A game-playing agent (MAX) searches two plies ahead: MAX chooses one of three moves (B, C, D), then MIN replies with one of three moves. Positions at depth 2 are scored by the evaluation function

$$\mathbf{Eval = 2 \times M + 1 \times Mo}$$

where M is the material advantage of MAX and Mo is the mobility advantage of MAX. The leaf features (M, Mo) are given for the leaves in left-to-right order:

MIN node	Leaf 1 (M, Mo)	Leaf 2 (M, Mo)	Leaf 3 (M, Mo)
B	(1, 1)	(5, 2)	(3, 2)
C	(1, 0)	(1, 2)	(2, 2)
D	(6, 2)	(2, 1)	(0, 2)

(a) Compute the Eval value of all nine leaves.

(b) Run plain minimax. Give the backed-up value of B, C and D, the value at the root and the move MAX should play.

(c) Run alpha-beta pruning on the same tree (left to right). Show α and β at each node, mark every pruned branch, and state the root value and the number of leaves evaluated.

(d) If the leaves under C were ordered (6, 4, 2) instead, how many leaves would alpha-beta evaluate? Explain the effect of move ordering and why pruning never changes the minimax value.

Question 4: Answer

(a) Calculate all nine leaf evaluations

Using:

$$\text{Eval} = 2M + Mo$$

Node B

Leaf 1: $(M, Mo) = (1, 1)$

$$E_1 = 2(1) + 1 = \boxed{3}$$

Leaf 2: $(5, 2)$

$$E_2 = 2(5) + 2 = \boxed{12}$$

Leaf 3: $(3, 2)$

$$E_3 = 2(3) + 2 = \boxed{8}$$

Node C

Leaf 1: $(1, 0)$

$$E_1 = 2(1) + 0 = \boxed{2}$$

Leaf 2: $(1, 2)$

$$E_2 = 2(1) + 2 = \boxed{4}$$

Leaf 3: $(2, 2)$

$$E_3 = 2(2) + 2 = \boxed{6}$$

Node D

Leaf 1: $(6, 2)$

$$E_1 = 2(6) + 2 = \boxed{14}$$

Leaf 2: $(2, 1)$

$$E_2 = 2(2) + 1 = \boxed{5}$$

Leaf 3: $(0, 2)$

$$E_3 = 2(0) + 2 = \boxed{2}$$

Final leaf values

MIN node	Leaf 1	Leaf 2	Leaf 3
B	3	12	8
C	2	4	6
D	14	5	2

(b) Run plain Minimax

In minimax:

- MIN selects the minimum value under each branch.
- MAX selects the maximum value among the resulting branch values.

Node B

$$B = \min(3, 12, 8)$$

$$\boxed{B = 3}$$

Node C

$$C = \min(2, 4, 6)$$

$$\boxed{C = 2}$$

Node D

$$D = \min(14, 5, 2)$$

$$\boxed{D = 2}$$

Root value

MAX selects the highest value:

$$\text{Root} = \max(3, 2, 2)$$

$$\boxed{\text{Root value} = 3}$$

Best move for MAX:

$$\boxed{B}$$

The optimal move is B because it gives MAX a backed-up value of 3, compared with 2 for both C and D.

(c) Alpha-Beta Pruning

Initial values:

$$\alpha = -\infty,\ \beta = +\infty$$

Here:

- $\alpha$is the best value found so far for MAX.
- $\beta$is the best value found so far for MIN.

Process the branches from left to right.

Step 1: Explore B

Evaluate the leaves in order:

$$B_1 = 3,\ B_2 = 12,\ B_3 = 8$$

MIN selects:

$$B = \min(3, 12, 8) = 3$$

At the root, MAX updates:

$$\alpha = \max(-, \infty 3) = 3$$

Step 2: Explore C

At C, the initial bounds are:

$$\alpha = 3,\ \beta = +\infty$$

First leaf:

$$C_1 = 2$$

MIN updates:

$$\beta = \min(+, \infty 2) = 2$$

Now:

$$\beta \leq \alpha$$
$$2 \leq 3$$

Therefore, the remaining leaves under C are pruned.

Pruned: $C_2 = 4$ and $C_3 = 6$.

The backed-up value of C is 2.

Step 3: Explore D

At D:

$$\alpha = 3,\ \beta = +\infty$$

First leaf:

$$D_1 = 14$$

Update:

$$\beta = \min(+, \infty 14) = 14$$

Since $\beta > \alpha$, continue.

Second leaf:

$$D_2 = 5$$

Update:

$$\beta = \min(14, 5) = 5$$

Since $\beta > \alpha$, continue.

Third leaf:

$$D_3 = 2$$

Update:

$$\beta = \min(5, 2) = 2$$

Now:

$$\beta \leq \alpha$$
$$2 \leq 3$$

The branch is complete; there are no further leaves to prune under D.

The backed-up value is:

$$D = \min(14, 5, 2) = 2$$

Alpha-Beta summary

Node	Leaf evaluations	Backed-up value	Pruned leaves
B	3, 12, 8	3	None
C	2	2	4, 6
D	14, 5, 2	2	None

Total leaves evaluated:

$$3 + 1 + 3 = \boxed{7}$$

Total leaves pruned:

$$\boxed{2}$$

Root value:

$$\boxed{3}$$

Best move: B.

Alpha-beta pruning produces the same minimax result while evaluating fewer leaves.

(d) If the leaves under C were ordered (6, 4, 2)

Assume the original values under B and D remain unchanged.

After evaluating B, the root has:

$$\alpha = 3$$

Now explore C with the leaf order:

$$6, 4, 2$$

First leaf:

$$C_1 = 6$$

Update:

$$\beta = 6$$

Since $6 > 3$, continue.

Second leaf:

$$C_2 = 4$$

Update:

$$\beta = \min(6, 4) = 4$$

Since $4 > 3$, continue.

Third leaf:

$$C_3 = 2$$

Update:

$$\beta = \min(4, 2) = 2$$

Now:

$$\beta \leq \alpha$$
$$2 \leq 3$$

There are no more leaves under C, so no leaf is pruned.

Therefore, all three leaves under C must be evaluated.

Total leaves evaluated:

- B: 3
- C: 3
- D: 3

$$\boxed{\text{Total} = 9 \text{ leaves}}$$

Effect of move ordering

Good move ordering can help alpha-beta pruning identify branches that cannot affect the final decision earlier, reducing the number of leaves evaluated.

Poor move ordering may delay pruning and cause more leaves to be evaluated.

However, **alpha-beta pruning never changes the minimax value** because it removes only branch that cannot affect the final decision, assuming the same evaluation values and search depth.